TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 1: Concurrency in Practice

Network Programming (PR)

Students: Gafenco Victor and Balan Gabriel

Group: FAF-241

Instructor: Assistant lecturer, Fiştic Cristofor

Chișinău - 2026

CONTENTS

Abstract **3**

Abbreviations **4**

Introduction **5**

1 Environment and measurement method **6**

1.1 Hardware and software 6

1.2 Input and correctness 6

1.3 Timing, memory, and aggregation 7

2 Task 1: Counting one dataset **8**

2.1 Task 1a: Independent file partitions 8

2.1.1 Scaling and processor resources 9

2.1.2 Memory and workload classification 9

2.2 Task 1b: Shared and local counters 10

2.3 Task 1c: Bounded queue and consumers 11

2.3.1 Why adding consumers has little effect 12

2.3.2 Comparison with independent partitions 12

2.3.3 Additional experiment: six readers 12

2.3.4 Batch size and queue capacity 14

3 Task 2: Files on USB storage **15**

3.1 Comparison and limiting resources 15

3.2 Memory cost 16

4 Control questions **17**

4.1 Physical cores and logical processors 17

4.2 Correctness at file boundaries 17

4.3 Lost updates in shared counters 17

4.4 Why local counters are faster 17

4.5 I/O-bound storage and network services 17

4.6 Static partitions versus a queue 18

Conclusions **19**

ABSTRACT

This laboratory investigates how work distribution and synchronization affect execution time, correctness, and resident memory when counting the signs of 50 million integers. We examine independent file partitions, three shared-array counting strategies, a bounded producer–consumer queue, and eight files read from USB storage. The saved measurements show a best speed-up of 2.857 for independent partitions and 1.092 for the single-reader queue implementation. Local counters complete in 269.903 ms, while a mutex around every shared update takes 11,213.016 ms. All three unsynchronized runs lose more than 35 million updates. The USB series improves from 11.174 s to 6.522 s with four threads. A separate six-reader experiment is retained for comparison: it has lower elapsed times, although the runs were collected in different sessions. Conclusions are restricted to the existing data; no new benchmark was executed for this report.

Keywords: concurrency, threads, synchronization, race condition, bounded queue, speed-up

ABBREVIATIONS

CPU	Central processing unit
I/O	Input/output
RSS	Resident set size
SMT	Simultaneous multithreading
USB	Universal Serial Bus
MiB	2^{20} bytes

INTRODUCTION

Concurrency allows several activities to progress during the same interval; parallelism executes work simultaneously on available processor resources. Adding threads can reduce elapsed time, but shared locks, storage bandwidth, allocation, and scheduling can limit the benefit. Our experiment uses one C++ program with command-line selections for each counting strategy. The work was completed jointly by Gafenco Victor and Balan Gabriel, group FAF-241.

We analyse the existing source, reference counts, CSV measurements, and supplied plots. The objective is to identify where parallelism helps, quantify its memory cost, and explain why correct synchronization and fast execution must be considered together. Numerical observations are distinguished from possible causes that would require profiling to establish.

1 ENVIRONMENT AND MEASUREMENT METHOD

1.1 Hardware and software

Table 1.1 records the available local environment and build metadata. This inventory was inspected when preparing the report; the CSV files do not embed a historical hardware or compiler inventory. It therefore describes the available workstation, without proving every setting at measurement time.

Table 1.1 – Available environment and evidence

Item	Recorded value
Processor	Intel Core i7-8650U, nominal 1.90 GHz
Execution resources	4 physical cores, 8 logical processors; 2 SMT threads per core
Memory	Approximately 15 GiB usable RAM reported by the operating system
Local storage	KINGSTON SNV2S1000G, NVMe SSD, 931.5 GiB
Operating system	openSUSE Tumbleweed, snapshot 20260901; Linux 7.2.7-1-default, x86-64
Language and compiler	C++23; cached compiler: Clang 23.1.2
C++ runtime	Installed libstdc++6 package: 16.2.1+git9713-1.1
Build configuration	Cache says Release; saved compilation flags contain no optimization flag
USB device and connection	Model, USB version, and negotiated link speed are not recorded in the supplied results

The saved flags include `-std=c++23` and warning options but no `-O2` or `-O3`. We cannot establish the measured binary's optimization level from a Release cache entry alone. The results should not be presented as verified optimized-build performance. Task 1b records eight threads, consistent with the workstation's logical processor count.

1.2 Input and correctness

The input file contains 50,000,000 integers in the range $[-1,000,000, 1,000,000]$, one per line. The generator uses `std::mt19937` with seed 42. The supplied file occupies 369,436,719 bytes (approximately 352.32 MiB). The reference and all correct-version CSV rows agree on the following counts:

Negative	24,994,850
Zero	26
Positive	25,005,124
Total	50,000,000

Comparing all three counters is stronger than checking only the sum: equal and opposite classification errors could preserve the total. The unsynchronized version is intentionally excluded from the correctness requirement. Task 2 uses eight consecutive files with 6,250,000 integers each.

1.3 Timing, memory, and aggregation

The benchmark launcher starts a fresh counting process for each measurement. The program uses `steady_clock`; timing includes task setup, thread creation, counting, joining, and reduction. For Task 1b, loading the integer array occurs before timing. Peak memory is the process-lifetime peak RSS, so it still includes that array. Launcher and plotting memory are excluded.

Tasks 1a and 1c have three runs per configuration. Before each run, the launcher reads the file without parsing to warm the cache. Their results therefore do not measure cold local-disk throughput. Task 1b has one local-counter run, one mutex run, and three race runs. Task 2 has one run at each thread count; its CSV policy is `manual-reconnect`. The launcher requests ejecting, unplugging, and reconnecting the USB before each run, without a warm-up read. The CSV records that policy but does not prove the physical procedure occurred.

Tables report median elapsed time and median peak RSS where repeats exist. Speed-up is computed from median times:

$$S(p) = \frac{\text{median}(T(1))}{\text{median}(T(p))}. \tag{1.1}$$

For the main Task 1c series, p means consumers, with one additional reader. The archived experimental series uses six readers and is identified separately. The charts show individual samples and observed minimum–maximum ranges, not confidence intervals. Task 1b compares strategies at a fixed eight threads; no single-thread Task 1b baseline exists, so a thread-scaling speed-up cannot be calculated for it.

2 TASK 1: COUNTING ONE DATASET

2.1 Task 1a: Independent file partitions

The coordinator obtains the file size and chooses approximately equal byte ranges. Each internal boundary is advanced through the next newline. The same boundary is the end of the preceding range and the start of the next range. Workers open their own file streams, seek to their starts, parse complete lines, and update local counters. The coordinator joins the workers and sums their results. It does not load or parse the entire file before distributing work.

The ranges are contiguous and half-open: $[b_i, b_{i+1})$. A worker reads a line only when its starting position precedes its end boundary. Thus the boundary line belongs to exactly one range. This reasoning assumes the generated newline-terminated input, for which byte positions advance by the line length plus one. Equal bytes need not mean exactly equal numbers of lines or equal execution time.

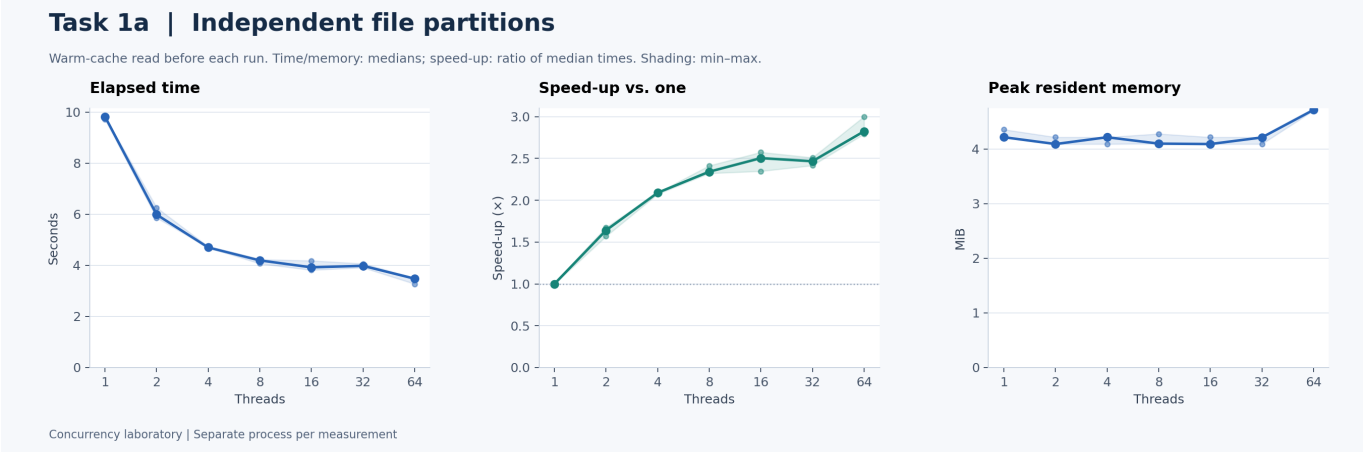


Figure 2.1 – Task 1a: elapsed time, speed-up, and peak resident memory

Table 2.1 – Task 1a medians from three runs per configuration

Threads	Time (s)	Speed-up	Peak RSS (MiB)
1	9.942	1.000	4.20
2	5.995	1.658	4.20
4	4.423	2.248	4.27
8	3.971	2.504	4.08
16	3.932	2.529	4.14
32	3.767	2.639	4.08
64	3.480	2.857	4.83

2.1.1 Scaling and processor resources

The largest improvements occur up to four threads, corresponding to the four physical cores. Moving from four to eight threads reduces median time by about 10.2%, much less than doubling throughput. Logical processors share a core's execution resources and do not provide the capacity of extra physical cores.

There is no strict stopping point in the measured range: the best median occurs at 64 threads. The curve flattens substantially around 8–16 threads, but 32 and 64 threads still improve these measurements. Relative to eight threads, 64 threads reduce median time by about 12.4%. We therefore cannot claim that 32 and 64 threads were slower. Extra threads cannot create extra cores, and context switching, contention, and cache pressure can eventually outweigh their benefit. The modest continuing improvement could involve scheduling or load balance; timings alone do not identify its cause. The eight-thread samples span 3.799–4.875 s, illustrating considerable run variability.

2.1.2 Memory and workload classification

Median RSS stays near 4.1–4.3 MiB through 32 threads and rises to 4.83 MiB at 64. The program streams its partitions instead of retaining all input. Additional thread stacks and stream buffers can increase memory, but reserved stack address space is not equivalent to resident memory. Small RSS differences also reflect allocation and page residency.

Warm-cache input and the substantial benefit of multiple cores are consistent with a CPU-side parsing workload rather than slow physical-disk reads being the dominant cost. This is an inference from the procedure and scaling, not a CPU-utilization or storage-throughput profile.

2.2 Task 1b: Shared and local counters

All variants scan the same preloaded integer array using eight threads. Version 1 updates three shared counters without synchronization. Version 2 locks a common mutex for every update. Version 3 keeps counters local to each worker and reduces them after joining.

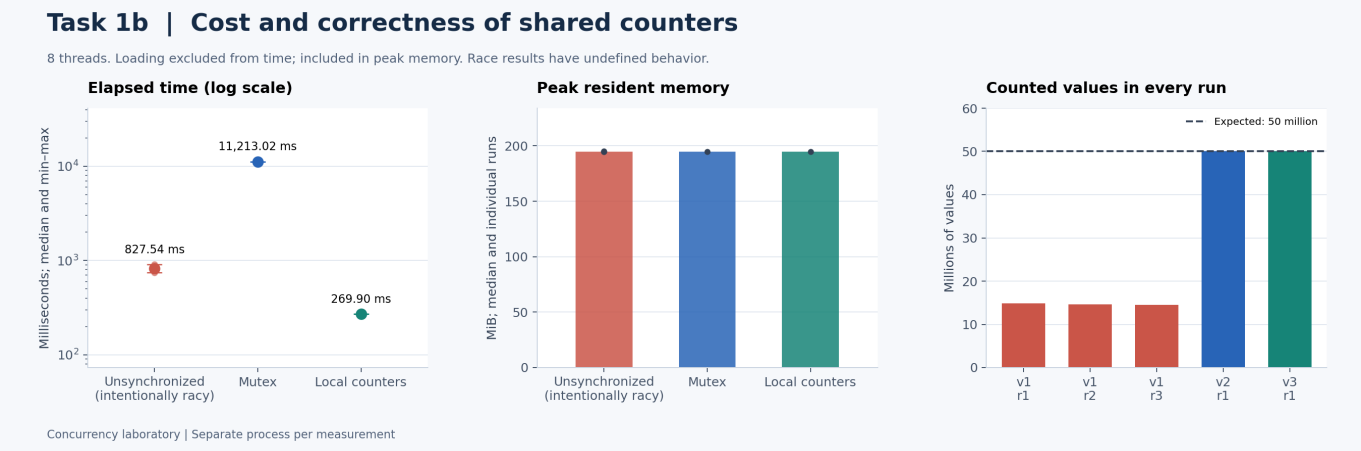


Figure 2.2 – Task 1b: strategy timing, peak memory, and correctness

Table 2.2 – Task 1b measurements; all configurations use eight threads

Strategy / run	Time (ms)	Peak RSS (MiB)	Counted total
Local counters	269.903	194.82	50,000,000
Mutex	11,213.016	194.65	50,000,000
Race, run 1	742.858	194.96	14,801,194
Race, run 2	827.543	194.68	14,601,519
Race, run 3	908.762	194.85	14,560,480

The measured ranking is local counters, unsynchronized counters, then mutex. The median race time is 827.543 ms, about 3.07 times the local-counter time. The mutex run takes about 41.54 times as long as the local-counter run. Only the mutex and local variants match all three reference counts. The race runs retain only about 29.1–29.6% of the expected total.

An increment is conceptually a read, an addition, and a write. If two threads read a counter at 10, both can compute 11 and both store 11; two updates have increased the counter by only one. This illustrates lost updates, but the C++ data race has undefined behavior, so the language guarantees neither this particular execution nor a particular wrong total. Even a matching total would not establish correctness.

The mutex preserves updates but serializes 50 million critical sections and requires repeated lock acquisition. Local counters remove this shared hot spot; only the final reduction combines results. Unsyn-

chronized shared writes can still incur cache-coherence traffic, explaining why avoiding a lock need not outperform local accumulation. The integer array alone occupies about 190.73 MiB when integers are four bytes; this accounts for most of the roughly 195 MiB peak RSS in every variant.

2.3 Task 1c: Bounded queue and consumers

The main experiment uses one reader and a bounded queue of at most 100 batches, with 10,000 raw lines per batch. The reader builds batches of strings; consumers remove batches, parse their lines outside the queue lock, and count locally. A mutex protects the queue and the remaining-reader counter. Condition variables block the reader when the queue is full and consumers when it is empty. Consumers exit only when the queue is empty and the reader has finished. The source now sets `READER_COUNT = 1`; the new CSV records total background threads as workers plus one, excluding the main coordination thread.

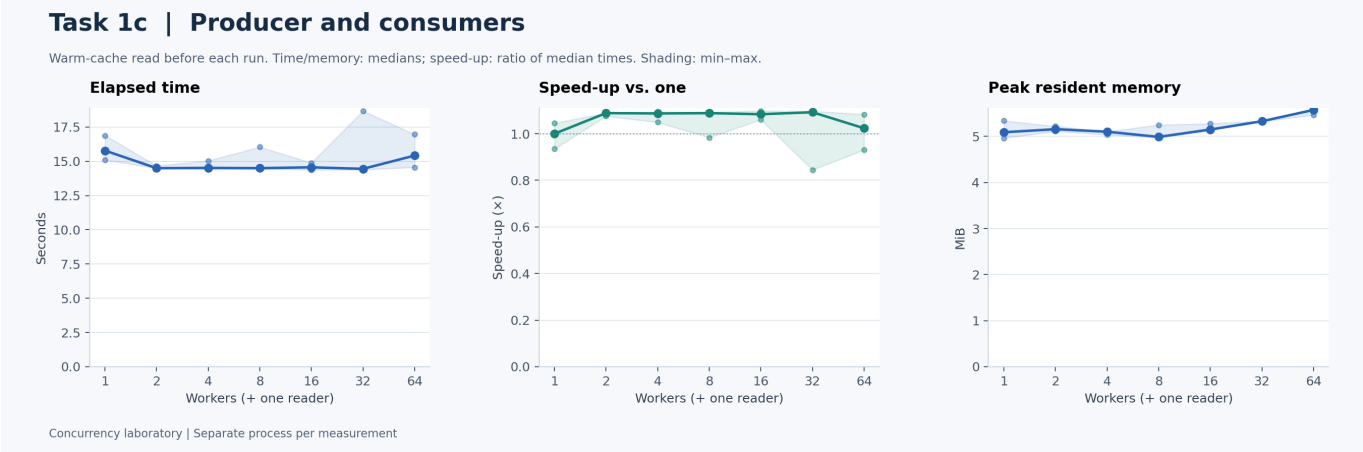


Figure 2.3 – Task 1c: one reader, variable consumers, and a bounded queue

Table 2.3 – Single-reader Task 1c medians from three runs per configuration

Workers	Time (s)	Speed-up	Peak RSS (MiB)
1	15.781	1.000	5.09
2	14.496	1.089	5.16
4	14.518	1.087	5.10
8	14.499	1.088	4.99
16	14.561	1.084	5.15
32	14.448	1.092	5.33
64	15.416	1.024	5.57

2.3.1 Why adding consumers has little effect

One consumer takes a median 15.781 s. Two consumers reduce this to 14.496 s, a speed-up of only 1.089. From two through 32 consumers, medians remain between 14.448 and 14.561 s. The lowest median is at 32 consumers, but its difference from two consumers is only about 0.34%; this does not establish a meaningful advantage. The practical plateau begins around two consumers. At 64 consumers, the median rises to 15.416 s. The 32-consumer samples span 14.385–18.678 s, showing why the minimum median should not be treated as a precisely located optimum.

The reader still performs 50 million line reads and constructs the batches of strings sequentially. Consumers only parse those short integer strings and increment local counters. Once consumers can keep up with production, adding more of them cannot increase the reader's rate. This is consistent with a producer bottleneck; it is not evidence that the laptop cannot run threads in parallel. Warm-cache input reduces physical storage reads but does not remove line handling, string operations, queue transfers, or scheduling costs. The inspected compilation flags also contain no optimization option, which is another plausible contributor to absolute execution time. We did not change build settings or collect a profiler trace, so individual costs are not isolated.

Median RSS remains between about 4.99 and 5.57 MiB, much lower than in the six-reader experiment. This is consistent with a queue that does not retain many batches because consumers keep up with the producer. Queue occupancy and thread utilization were not measured, so this remains an interpretation rather than a directly observed mechanism.

2.3.2 Comparison with independent partitions

Task 1a with one thread takes 9.942 s, compared with 15.781 s for Task 1c with one consumer and one reader. At four workers/threads, Task 1a takes 4.423 s versus 14.518 s for Task 1c. Their best recorded medians are 3.480 s and 14.448 s, respectively, a ratio of approximately 4.15. The runs were obtained in different sessions, so these are descriptive comparisons, not controlled estimates of the cost of changing only the algorithm.

Task 1a distributes both reading and parsing among workers. Task 1c leaves line reading and batch construction with one producer and adds string storage and queue coordination. This difference explains why extra consumers may have little work to do even when there are unused processor resources.

2.3.3 Additional experiment: six readers

We also tried six readers to investigate whether parallel production changes throughput. Each reader processes a disjoint, line-aligned file range and sends batches to the same bounded queue. The archived CSV records workers plus six background threads. This was an intentional extension of the main experiment. Its

plot was generated with labels that still say “one reader”; for this archived figure only, the actual reader count is six.

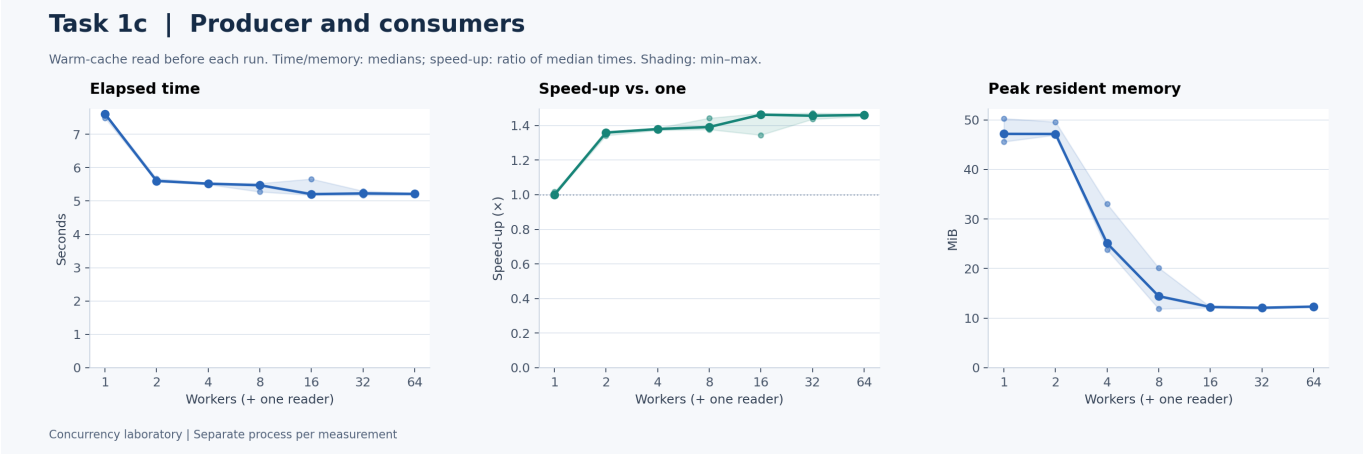


Figure 2.4 – Archived six-reader experiment; the image’s one-reader labels are stale

Table 2.4 – Recorded median times with one and six readers, at equal consumer counts

Workers	One reader (s)	Six readers (s)	Time ratio
1	15.781	7.614	2.07
2	14.496	5.602	2.59
4	14.518	5.520	2.63
8	14.499	5.473	2.65
16	14.561	5.206	2.80
32	14.448	5.228	2.76
64	15.416	5.213	2.96

These saved results do not show that adding readers makes no difference. At two consumers, the recorded median falls from 14.496 s with one reader to 5.602 s with six, a ratio of about 2.59. At 16 consumers, it falls from 14.561 s to 5.206 s, a ratio of about 2.80. The six-reader series itself gains little from adding consumers beyond two, with its best median at 16 consumers and essentially no further improvement at 32 or 64.

The distinction is between increasing the number of readers and increasing the number of consumers. The recorded six-reader times are substantially lower, while extra consumers have diminishing returns in both series. This is consistent with production limiting the single-reader design. However, the series were collected in separate sessions without a preserved, matched record of build flags, system load, temperature, or CPU frequency. Consequently the ratios do not establish the isolated causal benefit of adding five readers.

The archived peak RSS is about 47 MiB at one or two consumers and about 12 MiB at 16–64 consumers. Multiple producers can retain more text in the queue or in batches under construction. This agrees with increased production capacity, although queue occupancy was not logged.

2.3.4 Batch size and queue capacity

The 100-batch queue limit bounds queued work to one million lines, but batches under construction or being processed occupy additional memory. Single-line batches would require up to 50 million queue transfers instead of roughly 5,000 full batches, increasing locking, notification, and allocation overhead. An unbounded queue would remove backpressure: if production outpaces consumption, much of the input could accumulate as strings and potentially cause paging. A bounded queue controls that risk; its capacity is a maximum, not a promise that the queue will fill during a run.

3 TASK 2: FILES ON USB STORAGE

Each thread owns a fixed selection of the eight files. With one, two, or four threads, each processes eight, four, or two files respectively. A file is read using one stream read request into one byte buffer, then parsed directly from that buffer. The buffer is released before processing the next file. A single application read request may still require multiple operating-system or device transfers.

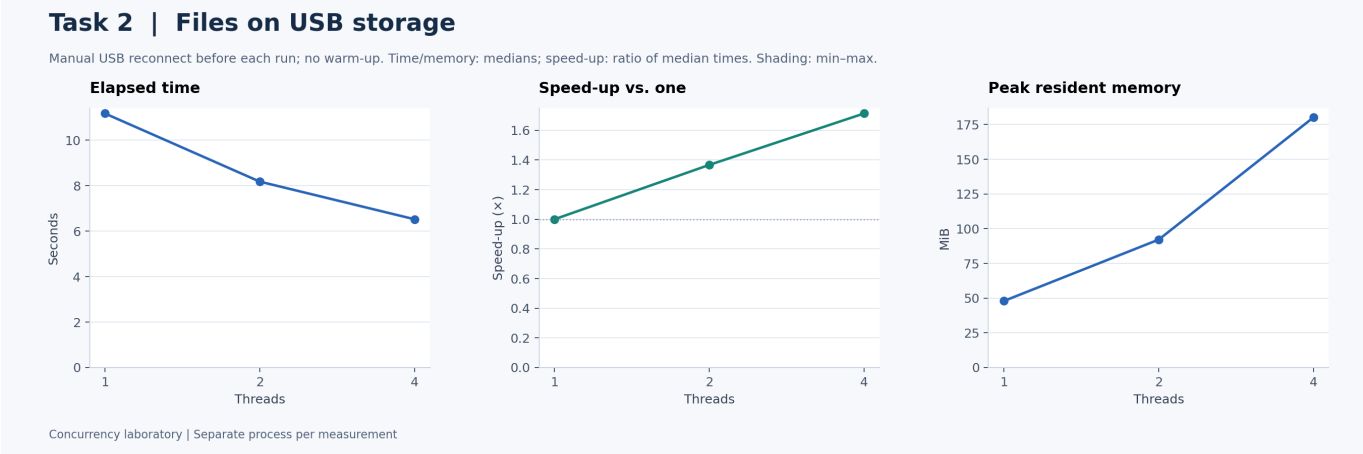


Figure 3.1 – Task 2: elapsed time, speed-up, and peak resident memory

Table 3.1 – Task 2 results; one measurement per configuration

Threads	Time (s)	Speed-up	Peak RSS (MiB)
1	11.174	1.000	47.97
2	8.177	1.366	92.16
4	6.522	1.713	180.22

3.1 Comparison and limiting resources

Task 2 on one thread takes 11.174 s, compared with 9.942 s for Task 1a on one thread: a difference of 1.231 s, or about 12.4%. Dividing total input bytes by elapsed time gives approximately 33.1 MB/s and 37.2 MB/s respectively. These are end-to-end processing rates, not isolated device read speeds.

The comparison changes both cache policy and reading strategy: Task 1a uses warm-cache line streaming, while Task 2 reads whole files under the recorded reconnect policy. It does not isolate USB latency. Moreover, the USB model and link version are absent. The available numbers therefore support a mixed storage-and-parsing workload, but do not conclusively establish that USB I/O alone dominates execution time.

More threads do help in this series: two threads reduce time by about 26.8% and four by about 41.6%, relative to one. Four threads achieve 1.713 times the throughput, well below ideal fourfold scaling. Parsing different buffers can proceed in parallel, and some computation can overlap reads. All reads still share the same USB device and connection, so concurrency cannot multiply their physical bandwidth. One sample per configuration limits conclusions about repeatability.

3.2 Memory cost

Peak RSS rises approximately with the number of simultaneous buffers: 47.97 MiB, 92.16 MiB, and 180.22 MiB. An average split file occupies about 44.04 MiB, so four live file buffers explain most of the four-thread peak. The implementation processes files sequentially within each worker, avoiding retention of all eight buffers at once.

4 CONTROL QUESTIONS

4.1 Physical cores and logical processors

A physical core contains execution hardware; SMT exposes more than one logical processor sharing that hardware. This workstation has four cores and eight logical processors. Task 1a improves strongly up to four threads, then only modestly from four to eight. This is consistent with shared core resources, although the measurements alone do not isolate SMT's benefit.

4.2 Correctness at file boundaries

Internal byte offsets are advanced to complete-line boundaries. Adjacent workers use the same boundary as an exclusive end and inclusive start. Every line therefore lies in one range. The matching negative, zero, and positive counters provide an empirical check across all recorded Task 1a configurations, in addition to the boundary reasoning.

4.3 Lost updates in shared counters

Two threads can read the same old value, independently increment it, and write the same new value. One increment is lost. The race experiment demonstrates incorrect totals in all three runs, but its undefined behavior prevents predicting exact results. A mutex makes each update mutually exclusive.

4.4 Why local counters are faster

Private counters avoid a lock on every element and repeated writes to the same shared cache lines. Workers synchronize at completion, then a small reduction adds three values per thread. This replaces synchronization proportional to the 50 million elements with a reduction proportional to the thread count.

4.5 I/O-bound storage and network services

An I/O-bound computation spends much of its elapsed time waiting for external data rather than using available CPU capacity. A single USB drive has shared bandwidth; additional readers eventually compete for the same resource. A server handling independent network requests can often use otherwise idle time to serve another request while the first waits for a remote response. Concurrency helps when there is independent work and unused capacity; it is not a guarantee for every I/O-bound system. Network bandwidth and remote services can also saturate. Our USB measurements show partial benefit, so they do not justify claiming that concurrency never helps USB access.

4.6 Static partitions versus a queue

Static partitioning assigns work before execution and avoids queue operations. It works well when partitions have similar costs, as in this sign-counting workload. A queue assigns the next batch to whichever consumer becomes free, which can handle uneven task costs more flexibly. That flexibility requires synchronization, batching, and memory for queued work. Here, independent partitions achieve the best absolute time, while the queue adds substantial memory and coordination costs.

CONCLUSIONS

The measurements show that workload structure matters more than the thread count alone. Independent partitions reach 2.857 times their one-thread throughput with little resident-memory growth. Most scaling occurs near the physical-core count, but the recorded medians continue improving through 64 threads; a complete plateau is not observed for Task 1a.

Local counters are both correct and fastest in the array experiment. A shared mutex preserves correctness at a large measured cost, while unsynchronized updates produce incorrect counts in every run. The single-reader queue reaches a practical plateau around two consumers and offers at most 1.092 times its one-consumer throughput in the saved medians. This is consistent with sequential production limiting consumer utilization. The additional six-reader experiment has substantially lower recorded times, but unmatched measurement sessions prevent attributing the entire difference to reader count alone.

Warm-cache file parsing and in-memory counting are consistent with CPU-side work and synchronization costs. The USB series benefits from extra threads but shows sublinear scaling and substantially greater memory use from whole-file buffers. Its data do not isolate a purely I/O-bound bottleneck. Missing USB identification, uncertain historical build flags, and the absence of resource profiling limit stronger machine-specific conclusions.